



TRUSTSEC

Smart Contract Audit

Morpho – BlueFallbackRolling

02/09/2026

Executive summary

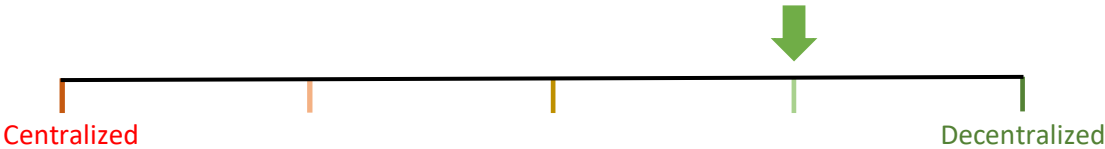


Category	Lending
Audited file count	2
Auditor	HollaDieWaldfee KlosMitSoss
Time period	19/08/2026 - 02/09/2026

Findings

Severity	Total	Fixed
High	0	0
Medium	0	0
Low	2	2

Centralization score



Signature



Digitally
signed by
Trust
Date:
2026.09.02
23:39:04
+03'00'

EXECUTIVE SUMMARY	1
DOCUMENT PROPERTIES	3
Versioning	3
Contact	3
INTRODUCTION	4
Scope	4
Repository details	4
About TrustSec	4
About the Auditors	4
Disclaimer	4
Methodology	5
QUALITATIVE ANALYSIS	6
FINDINGS	7
Low severity findings	7
TRST-L-1 Rolls can be grieved by debt reductions	7
TRST-L-2 Permissionless rollers can leave dust Midnight debt	7
Additional recommendations	9
TRST-R-1 Document borrow rounding alongside collateral rounding	9
TRST-R-2 Document Morpho Blue borrow share price assumption	9
TRST-R-3 Handle invalid debt amounts explicitly	9
Centralization risks	10
TRST-CR-1 BlueFallbackRolling inherits Morpho Midnight and Morpho Blue centralization risks	10
Systemic risks	11
TRST-SR-1 Chain outages can prevent configured rolls	11
TRST-SR-2 Supported token assumptions are inherited from Midnight and Morpho Blue	11
TRST-SR-3 BlueFallbackRolling depends on Midnight and Morpho Blue	11

Document properties

Versioning

Version	Date	Description
0.1	27/08/2026	Client report
0.2	02/09/2026	Mitigation review

Contact

Trust

trust@trust-security.xyz

Introduction

TrustSec has conducted an audit at the customer's request. The audit is focused on uncovering security issues and additional bugs contained in the code defined in scope. Some additional recommendations have also been given when appropriate.

Scope

- `src/periphery/blue-fallback-rolling/BlueFallbackRolling.sol`
- `src/periphery/blue-fallback-rolling/interfaces/IBlueFallbackRolling.sol`

Repository details

- **Repository URL:** <https://github.com/morpho-org/midnight>
- **Commit hash:** 14e0b778434ae93fb87385f41e76136947612673
- **Mitigation hash:** 12503f275589d273bc39eb4b5a8460b8bbbed4c18

About TrustSec

TrustSec has been established by top-end blockchain security researcher Trust, in order to provide high quality auditing services. Trust is the leading auditor at competitive auditing service Code4rena, reported several critical issues to Immunefi bug bounty platform and is currently a Code4rena judge.

About the Auditors

HollaDieWaldfee is a distinguished security expert with a track record of multiple first places in competitive audits. He is a Lead Auditor at TrustSec and Senior Watson at Sherlock.

KlosMitSoss is a security researcher with a strong competitive auditing track record, including multiple first place finishes on Code4rena. He has extensive experience conducting both private security reviews and public audit competitions.

Disclaimer

Smart contracts are an experimental technology with many known and unknown risks. TrustSec assumes no responsibility for any misbehavior, bugs or exploits affecting the audited code or any part of the deployment phase.

Furthermore, it is known to all parties that changes to the audited code, including fixes of issues highlighted in this report, may introduce new issues and require further auditing.

Methodology

In general, the primary methodology used is manual auditing. The entire in-scope code has been deeply looked at and considered from different adversarial perspectives. Any additional dependencies on external code have also been reviewed.

Qualitative analysis

Metric	Rating	Comments
Code complexity	Excellent	<i>BlueFallbackRolling</i> is a minimal and simple periphery contract.
Documentation	Good	While <i>BlueFallbackRolling</i> is easy to understand, its documentation could be more detailed.
Best practices	Excellent	Project adheres to best practices.
Centralization risks	Excellent	<i>BlueFallbackRolling</i> does not introduce additional centralization risks.

Findings

Low severity findings

TRST-L-1 Rolls can be grieved by debt reductions

- **Category:** Griefing attacks
- **Source:** BlueFallbackRolling.sol
- **Status:** Fixed

Description

BlueFallbackRolling.roll() requires the caller to provide an exact **assets** amount. A caller intending to roll the full Midnight debt must therefore determine **assets** from the user's current debt before submitting the transaction.

Before execution, another permissionless roll or a valid liquidation can reduce the user's Midnight debt. The submitted **assets** value then exceeds the newly read **debtAssets** value, and the subsequent *Midnight.repay()* call reverts when it subtracts more debt than remains. A roller or liquidator can therefore invalidate a pending roll and force its caller to resubmit.

Recommended mitigation

It is recommended to support a full-roll sentinel such as **assets=type(uint256).max** that rolls all of the user's debt. Exact values should remain unchanged for partial rolls.

Alternatively, document that roller integrations must account for this griefing risk. External roller contracts should read the live Midnight debt and call *roll()* atomically so that competing rolls or liquidations cannot stale the amount between sizing and execution.

Team response

Fixed via documentation at commit [50db1bb](#).

Mitigation review

The finding has been addressed via documentation, and it has been determined that it is the responsibility of an external roller contract to size **assets** atomically.

TRST-L-2 Permissionless rollers can leave dust Midnight debt

- **Category:** Griefing attacks
- **Source:** BlueFallbackRolling.sol
- **Status:** Fixed

Description

BlueFallbackRolling.roll() is permissionless and accepts a partial roll whenever **assets** $\geq$ **minRollableAssets**, without constraining the remaining **debtAssets** - **assets**. A roller can therefore migrate almost the entire debt while deliberately leaving a nonzero dust remainder.

The roller receives the configured incentive on nearly the full amount and gives up only the incentive on the dust.

Although the **assets** $=$ **debtAssets** condition technically allows a final roll below **minRollableAssets**, the incentive factor is capped at **WAD**, and therefore the reward for clearing a very small remainder cannot exceed its value and may not cover gas and other costs. The griever can thus prevent the fallback migration from completing while still earning the incentive on almost all debt.

Recommended mitigation

It is recommended to add a user-configured **minDebtRemaining** value to the rolling configuration. After explicitly validating **assets** $\leq$ **debtAssets**, a roll should only be allowed when it repays the entire debt or when **assets** $\geq$ **minRollableAssets** and **debtAssets** - **assets** $\geq$ **minDebtRemaining**. The condition thus becomes **require((assets $\geq$ minRollableAssets && (debtAssets - assets) $>$ minDebtRemaining) || assets $=$ debtAssets, RolledAssetsTooLow());**. This new value should be included in the configuration identifier and **SetConfig** event.

If the same threshold is suitable for both purposes, **minRollableAssets** can instead be applied to both the amount rolled and the debt remaining after a partial roll.

Team response

Fixed via documentation at commit [7a917f2](#).

Mitigation review

The finding has been addressed via documentation.

Additional recommendations

TRST-R-1 Document borrow rounding alongside collateral rounding

BlueFallbackRolling.roll() already [explains](#) that splitting a migration can increase the final Morpho Blue loan-to-value ratio because the collateral calculation rounds in favor of the Midnight position. Each split also executes a separate asset-denominated *IMorpho.borrow()* call, which rounds up the resulting borrow shares against the user. It is recommended to add a corresponding comment immediately before this *borrow()* call which explains that splitting can also increase the final Morpho Blue loan-to-value ratio through borrow-share rounding and that **minRollableAssets** is the intended mitigation.

TRST-R-2 Document Morpho Blue borrow share price assumption

BlueFallbackRolling borrows an asset-denominated amount from Morpho Blue without checking the resulting borrow shares or borrow share price. Morpho Blue [documents](#) that the borrow share price of a market with fewer than **1e4** assets borrowed can be manipulated downward until **totalBorrowShares** becomes very large and new borrows overflow. Add a contract-level comment stating that users must select Morpho Blue markets protected against such a manipulation, because it can block the borrow performed during a roll.

TRST-R-3 Handle invalid debt amounts explicitly

When **debtAssets == 0**, the collateral calculation divides by zero and reverts with **Panic(0x12)**. Similarly, the contract accepts a roll with **assets > debtAssets**, causing the later *Midnight.repay()* call to subtract more debt than remains and revert with **Panic(0x11)**. It is best practice to validate that **debtAssets > 0** and **assets <= debtAssets** after reading the live debt so invalid calls revert immediately before arithmetic or external calls are performed.

Centralization risks

TRST-CR-1 BlueFallbackRolling inherits Morpho Midnight and Morpho Blue centralization risks

BlueFallbackRolling does not introduce any centralization risks. As a periphery contract, it nevertheless inherits the centralization risks and governance assumptions of Midnight and Morpho Blue.

Systemic risks

TRST-SR-1 Chain outages can prevent configured rolls

BlueFallbackRolling relies on permissionless callers executing a configured roll before the Midnight position is liquidated. Even if a roll is configured correctly and its incentive makes execution profitable, a chain outage can prevent execution. Once the chain resumes, the user may be liquidated on Midnight before the position can be rolled.

TRST-SR-2 Supported token assumptions are inherited from Midnight and Morpho Blue

BlueFallbackRolling transfers and approves the Midnight market's loan and collateral token while composing calls to Midnight and Morpho Blue. The selected tokens must therefore satisfy the supported-token assumptions of both protocols. A token that is not supported by either protocol can cause rolls to revert or invalidate the accounting and transfer assumptions of the composed flow.

TRST-SR-3 BlueFallbackRolling depends on Midnight and Morpho Blue

BlueFallbackRolling is a periphery contract that composes Midnight and Morpho Blue. Its security and availability depend on the correctness and continued operation of both protocols. An exploit or failure in any integrated component can cause rolls to revert or expose users to losses even if *BlueFallbackRolling* behaves as intended.